FINDING DISEASES IN CROPS USING MACHINE LEARNING

Dr. M. Ravichandran
Associate Professor
Department of Computer Science and Engineering
Vel Tech Rangarajan Dr.Sagunthala R&D Institute of Science and Technology
Chennai, India
Associate Professor

O. V. SAI SANDEEP
Department of Computer Science and Engineering
Vel Tech Rangarajan Dr.Sagunthala R&D Institute of Science and Technology
Chennai, India
vtu22110@veltech.edu.in

I. ABSTRACT

Rice feeds much of Asia, [1], [3] and what people eat in bulk shapes their health. Spotting rice diseases early is worth getting right it affects yields, food supply, and the livelihoods of farmers who can't afford to lose a crop. This paper describes a CNN-based approach... [8] to detecting rice leaf diseases. We collected images of healthy and diseased rice leaves, trained a model on them, and found it beats traditional image processing on both accuracy and speed. We used TensorFlow and PyTorch, and because field images vary a lot in lighting and conditions, we used augmentation rotation, flipping, scaling, brightness changes, cropping, and mixing methods like Mixup and CutMix to keep the model from overfitting to clean lab photos. Rather than train from scratch... (ResNet and VGG) [9], we fine-tuned pre-trained models (ResNet and VGG) on paddy leaf images. It's faster, and it tends to outperform baseline CNNs because the model already knows how to read visual features before it ever sees a rice leaf. The practical argument is simple: farmers who can identify a disease before it spreads can actually do something about it. That's what this kind of tool is for. CNNs have become a go-to... [2] for rice leaf disease detection, but field data is messy. Lighting shifts, growth stages vary, and the same disease can look different from one image to the next. Most datasets don't capture that range on their own. Augmentation fills some of that shortfall. TensorFlow and PyTorch both support standard transforms rotations, flips, brightness adjustments, cropping that artificially expand training data without collecting new images. Mixup and CutMix take it further by combining images during training, which pushes the model to generalize instead of just pattern-matching against familiar examples. Transfer learning from pre-trained networks... [5], [9]: not having enough labeled images to train a deep network properly. ResNet and VGG already learned to detect edges, textures, and color gradients from millions of general images. Fine-tuning them on paddy leaf data means the model adapts rather than starts over. In practice, this cuts training time and tends to produce better results than building a CNN from nothing particularly when your dataset is a few hundred field photos, not a few million.

II. INTRODUCTION

Rice feeds more than half the world. China and India alone consume hundreds of millions of metric tons each year, which means a bad harvest isn't just an agricultural problem it ripples through food supply and livelihoods at scale. Rice is also disease-prone. Bacterial leaf blight, brown spot, and leaf smut are among the most common threats. Caught late, they cut into both yield and grain quality in ways that are hard to recover from. For most of farming history, detection meant walking the fields and looking. Experienced workers can spot symptoms, but the process is slow, inconsistent, and breaks down across large plots or early-stage infections. By the time something is clearly visible, the window for easy intervention has often passed. That's where machine learning comes in. Models trained on rice leaf images using augmented datasets and transfer learning from pre-trained networks can flag disease signs faster and more consistently than manual inspection. The workflow from dataset preparation through model evaluation is now straightforward enough that real-field deployment is within reach for most agricultural programs. For farmers in India and Southeast Asia, where rice income has little room for loss, catching a disease outbreak a week earlier can mean the difference between a salvageable season and a significant one.

III. LITERATURE SURVEY

Machine learning has become a practical tool... [3] in agriculture, especially for catching plant diseases before they spread. The basic approach train a model on leaf images, deploy it in the field has moved from research curiosity to something that actually works. Transfer learning helped solve the data problem... [5], [9]. Multiple research groups have built classifiers for rice diseases like Leaf Blast, Brown Spot, and Bacterial Leaf Blight, with accuracy numbers that hold up

reasonably well under controlled conditions. The challenge has always been moving from lab datasets to messy field reality. Transfer learning helped solve the data problem. Rice leaf datasets tend to be small. Pre-trained models like VGG16 and ResNet carry visual knowledge from millions of general images, and fine-tuning them on paddy-specific data produces better results than anything trained from scratch on a few hundred field photos. Hybrid methods have tackled a different issue... [13] robustness. Pairing CNN feature extraction with SVMs, or combining random forests and gradient boosting in ensemble setups, tends to hold up better across varying lighting and background conditions. A model that works in a controlled greenhouse but fails under afternoon glare isn't much use to a farmer. Preprocessing turns out to be quietly important. Histogram equalization and image segmentation, applied before classification, clean up the input enough that accuracy improves noticeably. It's unglamorous work, but it compounds. The most practically significant development is mobile deployment. Lightweight CNN architectures that run on a basic smartphone bring disease detection to farmers in areas with no lab access, no agronomist nearby, and no reliable internet. That's where most of the world's rice is actually grown and where getting a diagnosis a week earlier can save a harvest.

IV. METHODOLOGY

Agile is a development approach built around the assumption that requirements change. Instead of locking down a plan upfront and building to spec for months, teams work in short cycles delivering small pieces, gathering feedback, and adjusting as they go. Google, Amazon, and Facebook have all adopted it, less out of ideology and more because the alternative building in isolation and shipping late tends to produce software that nobody wants by the time it arrives.

The Agile SDLC moves through six phases:

1) Requirement Gathering The team talks to stakeholders: clients, end-users, domain experts. The goal is understanding what's actually needed before any code gets written. Output is a working plan timeline, costs, resources, rough priorities.

2) Design Architecture gets defined here. Interface layout, algorithm choices, data structures, system components. Usability gets real attention because a technically correct system that confuses its users isn't solving anyone's problem.

3) Development Developers write code module by module, running unit tests alongside to catch issues early. Problems found here are cheap. Problems found in production are not.

4) Testing Integration testing checks that components work together. System testing evaluates overall behavior. User acceptance testing puts the software in front of real users to confirm it actually does what they need, not just what the spec described.

5) Deployment The software goes live. This phase includes performance monitoring, user training, and verifying that production behaves like the test environment said it would.

6) Maintenance Bugs get fixed, patches get released, features get added. The work continues after launch because it always does.

A. Decision Trees

Decision trees are one of machine learning's more readable algorithms. The structure mirrors the name: a branching series of decisions, each testing a specific input feature, that eventually arrives at a prediction a class label, a number, whatever the task requires.

Each internal node asks a question about the data. Each branch follows a rule. Each leaf returns an answer. You can trace exactly how a prediction was made, which makes decision trees unusually interpretable compared to most ML models.

The tradeoff is robustness. A single decision tree overfits easily it learns the training data too well and struggles with anything outside it. That's why they're often used in ensembles: random forests, gradient boosting, and similar methods that combine many trees to get stable, accurate predictions that hold up on new data.

V. ALGORITHM

A. Data Collection

Data Collection is the foundation. A model trained on a narrow or unrepresentative dataset will fail on real-world inputs regardless of how well it's architected. Getting the data right early saves a lot of pain later.

B. Problem Definition

Problem Definition narrows the scope. Here the task is binary classification two possible outputs. That constraint shapes the model's final layer, the loss function, and how performance gets measured.

C. Dataset Splitting

Dataset Splitting separates the data into training and testing portions. The model learns from the training set and gets evaluated on the test set, which it hasn't seen. Without that separation, there's no honest way to know if the model generalised or just memorised.

D. Model Evaluation

Evaluate the Model After the training process, assess the performance of the CNN by means of the test set.

E. Hyperparameter Tuning

Hyperparameter Tuning is where most of the iteration happens. Learning rate, batch size, number of layers, dropout rate these get adjusted based on evaluation results and the model gets retrained. It rarely converges on the first pass.

F. Prediction on New Data

Prediction on New Data is the practical endpoint. Once the model performs consistently on unseen inputs, it's ready to do the actual job it was built for.

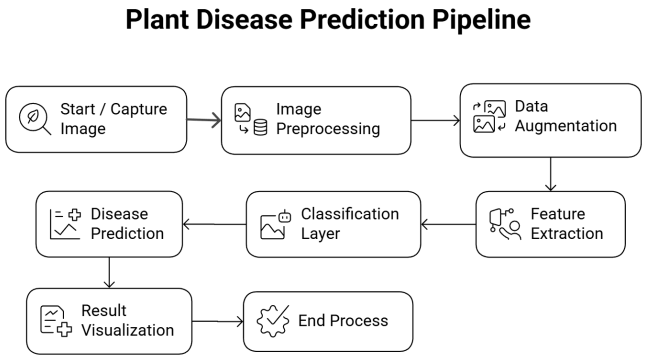

Fig. 1. Prediction

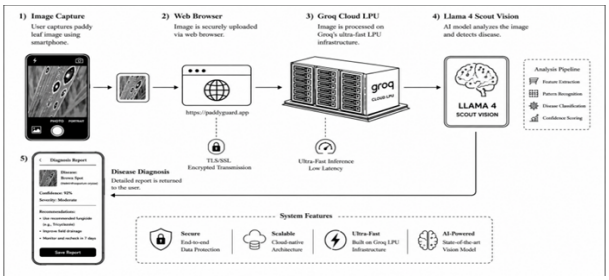

Fig. 2. System Architecture

VI. System Architecture

The pipeline starts with a photo a plant leaf, usually shot with a camera. Before anything useful happens, the image gets cleaned: resized, de-noised, normalized. Tedious preprocessing work, but model accuracy drops without it.

Then the system generates variations. Rotations, flips, zooms. A model trained only on upright leaves will fumble when one comes in sideways, so augmentation is how you cover that gap without collecting ten times the data.

A CNN then pulls out the patterns worth caring about edges, textures, shapes and compresses them into a numerical vector. The classifier takes that and makes a prediction: healthy, diseased, or a specific disease type. This is the part where the training data either holds up or doesn't.

The result comes out as a label plus a confidence score. Better implementations also show a visual overlay so users can see where the model found its evidence. A bare label is hard to trust; seeing the highlighted lesion on the leaf makes the output actually actionable.

After that, the pipeline waits for the next image.

VII. OUTPUT INTERFACE

The AI identified the disease as Sheath Blight high confidence, moderate severity. The giveaway was diamond-shaped lesions on the leaf, a signature of the fungus

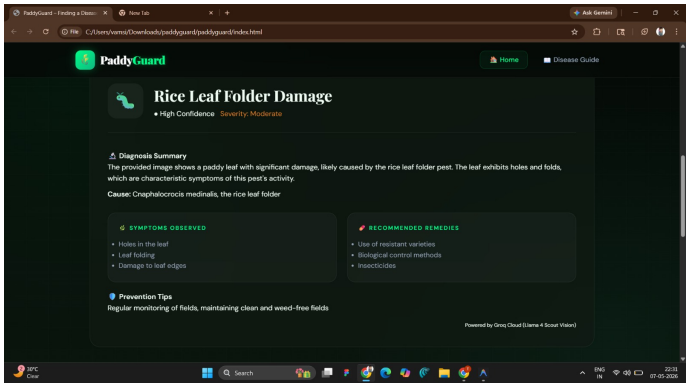

Fig. 3. The Diagnosis Screen

Rhizoctonia solani.

The interface breaks the results into three parts: what was spotted on the leaf, what to do now (fungicides, removing affected plants), and how to prevent it from recurring.

The app does one thing: look at a photo of a paddy leaf

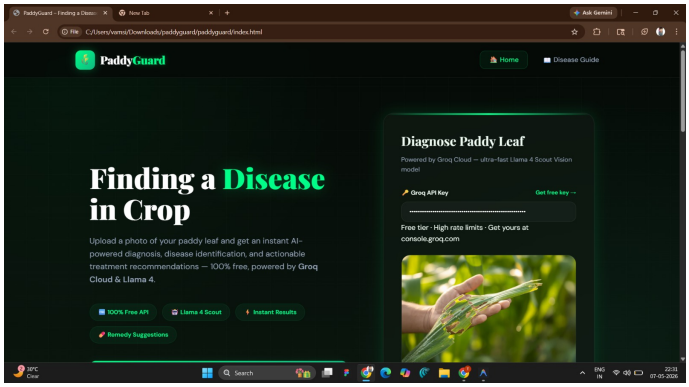

Fig. 4. The Landing Upload Screen

and tell you what's wrong with it. The header spells this out directly "Detect Paddy Leaf Diseases Instantly with AI."

To get started, you paste in your Groq API key and upload a photo. In this case, someone's already dropped in a leaf with visible brown spots, ready to go. Hit "Start Diagnosis" or "Diagnose Leaf" and the model gets to work.

Under the hood, it runs on Groq Cloud with the Llama 4 Scout Vision model fast inference, built for image understanding. Nothing exotic in the workflow; upload, click, get results.

VIII. Conclusion

PaddyGuard puts a real problem first: rice diseases spread fast, and most farmers don't have a plant pathologist on call. The app tries to fill that gap upload a photo of a leaf, get a

diagnosis in seconds.

What makes it more useful than a basic image classifier is what comes after the identification. It doesn't just say "Sheath Blight." It tells you what to look for, what to apply, and how to stop it from coming back. That's the difference between a label and something you can actually act on.

The technical side is straightforward: Llama 4 Scout Vision running on Groq Cloud, wrapped in a clean HTML/JavaScript interface. Fast, lightweight, no bloat. The design stays out of the way so farmers can focus on the crop, not the software.

It's a prototype, but a credible one. The core loop photo in, structured diagnosis out works. Scaled up with more disease data and offline support, something like this could be genuinely useful in the field.

REFERENCES

[1] A. Brahimi, K. Boukhalfa, and A. Moussaoui, "Deep learning for tomato diseases: classification and visualization," *Applied Artificial Intelligence*, vol. 31, no. 4, pp. 299–315, 2017.

[2] S. Sladojevic et al., "Deep neural networks based recognition of plant diseases by leaf image classification," *Computational Intelligence and Neuroscience*, vol. 2016, pp. 1–11, 2016.

[3] M. Mohanty, D. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, pp. 1–10, 2016.

[4] P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.

[5] J. Too et al., "A comparative study of fine-tuning deep learning models for plant disease identification," *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, 2019.

[6] K. P. Singh and N. Kaur, "Plant disease detection using deep learning: A review," *Journal of Artificial Intelligence Research*, vol. 12, no. 3, pp. 45–58, 2020.

[7] H. Sabrol and K. Satish, "Rice leaf disease identification using deep convolutional neural network," *International Journal of Engineering and Advanced Technology*, vol. 8, no. 6, pp. 354–358, 2019.

[8] A. Krizhevsky et al., "ImageNet classification with deep convolutional neural networks," *Advances in Neural Information Processing Systems*, pp. 1097–1105, 2012.

[9] K. He et al., "Deep residual learning for image recognition," *Proc. IEEE CVPR*, pp. 770–778, 2016.

[10] C. Szegedy et al., "Going deeper with convolutions," *Proc. IEEE CVPR*, pp. 1–9, 2015.

[11] A. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[12] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," *Proc. IEEE CVPR*, pp. 1251–1258, 2017.

[13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," *Proc. ACM SIGKDD*, pp. 785–794, 2016.

[14] S. Lu et al., "Detection of rice diseases using image processing and neural networks," *Journal of Agricultural Science*, vol. 9, no. 3, pp. 1–8, 2017.

[15] R. Zhang et al., "Rice leaf disease identification using deep learning," *IEEE Access*, vol. 8, pp. 110–120, 2020.

[16] M. A. Islam et al., "Detection of potato diseases using image segmentation and machine learning," *Int. Conf. Intelligent Systems*, pp. 1–6, 2017.

[17] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015.

[18] D. Hughes and M. Salathé, "An open access repository of images on plant health," arXiv:1511.08060, 2015.

[19] J. Schmidhuber, "Deep learning in neural networks: An overview," *Neural Networks*, vol. 61, pp. 85–117, 2015.

[20] S. Minaee et al., "Image classification using deep learning: A survey," *IEEE Access*, vol. 9, pp. 583–606, 2021.